MCMP117 · Week 11 · Course Notes

Fundamentals of Operating Systems

Permissions, Ownership & Users

Who may read, change or run your files — and how does Linux decide?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain why an operating system needs file permissions and how the kernel enforces them (links to Week 2 and Week 7).
2. Find your user name, UID, GID and groups with `whoami`, `id` and `groups`, and read the fields of `/etc/passwd` and `/etc/group`.
3. Decode a mode string such as `-rwxr-x---` and convert it to octal (750) and back.
4. Explain what r, w and x mean for files *and* for directories, and which of owner, group and others applies to a process.
5. Change permissions with `chmod` in symbolic (`u+x`, `go-w`) and octal (`640`) form, and make a script executable.
6. Compute default permissions from the `umask` correctly (bitwise, not by subtraction).
7. Explain `chown`, `chgrp`, `sudo`, the setuid bit of `passwd` and the sticky bit of `/tmp`, and read the related error messages.

Contents

1 From Week 10 to Week 11 1

2 Why Permissions? Protection Revisited 2

3 Users and Groups 2

4 Reading Permissions 4

5 Changing Permissions with `chmod` 5

6 Default Permissions: the `umask` 8

7 Directory Permissions in Practice 10

8 Ownership: chown and chgrp 11

9 root and sudo 12

10 Special Bits: setuid and the Sticky Bit 13

11 Command Reference 14

12 Common Errors and What They Mean 15

13 Summary 15

14 Glossary 16

15 Lab Sheet: Secure a Project Folder 16

16 Review Questions 17

17 Looking Ahead: Week 12 18

18 Lab Sheet Solutions 19

1 From Week 10 to Week 11

Last week you created, copied, moved and deleted files (`mkdir`, `touch`, `cp`, `mv`, `rm`), let the shell expand wildcards, and looked at files with `cat`, `less`, `head`, `tail` and `stat`. Every `ls -l` and `stat` printed something like `-rw-rw-r--` and `Access: (0664/-rw-rw-r--)`, together with the names of an owner and a group. This week we learn to read these fields, to change them, and to understand the users and groups behind them.

How to follow the examples. Re-create the practice folder first (it deletes and re-creates only `~/lab`):

```
$ bash mcmp117_lab_setup.sh
```

Every command in these notes was run on a Linux machine as an *ordinary* user called `student` (host `okan-lab`); the scripts `code/demo_*.sh` contain the exact commands and `code/out_*.txt` the captured output. On your machine dates, sizes, user and group *numbers* and names will differ, and the order of `ls` output can depend on the locale (we used `LC_ALL=C.UTF-8`). Only where the topic is `sudo` or `root` do we say explicitly that an administrator prepared something.

2 Why Permissions? Protection Revisited

2.1 The theory behind it

In **Week 2** we saw **dual-mode operation**: the CPU runs user programs in *user mode*, where privileged instructions such as direct I/O are forbidden. A program that wants to read a file cannot talk to the disk itself; it must make a **system call** such as `open()`, which switches to kernel mode. In **Week 7** we listed *protection* as one of the attributes of a file and saw that Linux stores it in the file's **inode**, together with the owner, the group, the size and the timestamps.

Put these two facts together and you have the whole mechanism:

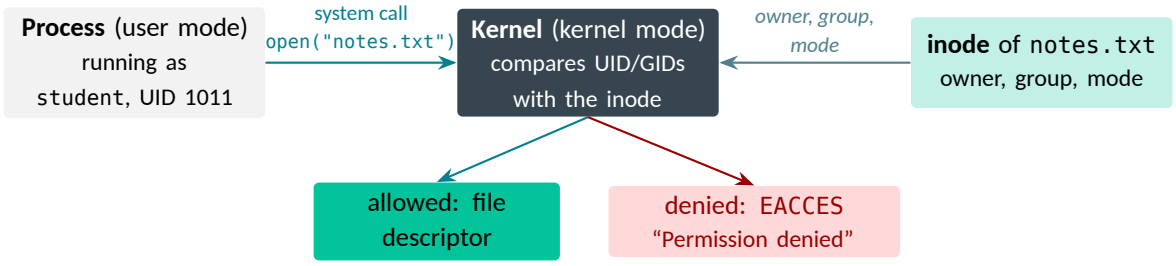


Figure 1: Every access goes through a system call, so the kernel can check it and it cannot be skipped.

Because the program *has* to ask the kernel, the kernel can refuse. If the check fails, the system call returns -1 and sets the error code EACCES; programs print it as *Permission denied*. The same check happens when you run a program (`exec`), enter a directory (`chdir`) or delete a name (`unlink`). Note that the check is made when the file is *opened*: a process that already has a file open keeps its access even if the permissions change afterwards.

> **Historical note**
>
> UNIX, designed at Bell Labs from 1969, was a multi-user time-sharing system from the start (Week 1), so it needed a cheap protection scheme. Instead of a full list of users per file (an *access control list*), Ken Thompson and Dennis Ritchie chose three classes — owner, group, others — with three bits each: nine bits in the inode. The scheme was so simple and effective that it survives unchanged in Linux and macOS. (Modern Linux also supports ACLs with `getfacl`/`setfacl`, but the nine bits remain the basis.)

3 Users and Groups

3.1 Who am I?

Linux knows two kinds of identity:

- A **user** has a login name and a number, the **UID** (user ID). The kernel stores and compares only the number. The user `root` has UID 0 and is the **superuser**. Many *system users* (for example `daemon` or `www-data`) exist only to run services with limited rights.
- A **group** is a named set of users with a number, the **GID**. Each user has exactly one *primary group* and may belong to several *supplementary groups*. Ubuntu creates a private group with the same name for every user.

Every process carries the UID and GIDs of the user who started it (its *credentials*); a file it creates gets that UID as owner and the primary GID as group.

```
student@okan-lab:~/lab$ whoami
student
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student)
student@okan-lab:~/lab$ groups
student
```

Here `uid=1011(student)` is the user number and name, `gid=1013(student)` the primary group, and `groups=` lists every group. On a freshly installed Ubuntu the first user normally has UID 1000; our lab machine has more accounts.

3.2 The account files `/etc/passwd` and `/etc/group`

```
student@okan-lab:~/lab$ grep "^root:" /etc/passwd
root:x:0:0:root:/root:/bin/bash
student@okan-lab:~/lab$ grep "^$USER:" /etc/passwd
student:x:1011:1013::/home/student:/bin/bash
student@okan-lab:~/lab$ grep "^$USER:" /etc/group
student:x:1013:
student@okan-lab:~/lab$ getent group students
students:x:1010:
student@okan-lab:~/lab$ ls -l /etc/passwd /etc/group /etc/shadow
-rw-r--r-- 1 root root    809 Oct 12 11:50 /etc/group
-rw-r--r-- 1 root root   1598 Oct 12 11:50 /etc/passwd
-rw-r----- 1 root shadow  884 Oct 12 11:50 /etc/shadow
student@okan-lab:~/lab$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

A line of `/etc/passwd` has seven fields separated by colons:

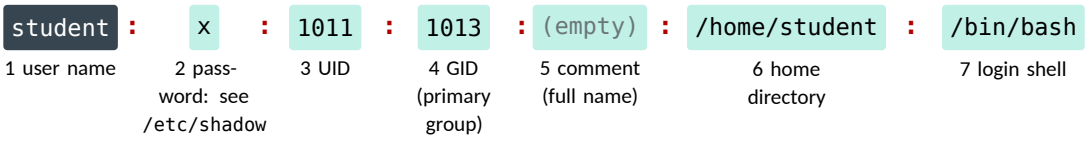


Figure 2: The seven fields of a line of `/etc/passwd`.

A line of `/etc/group` has four fields: group name, password (unused, x), GID and a comma-separated list of members. The group `students` has no members yet (empty fourth field).

Why is the password not in `/etc/passwd`? Every program must be able to *read* `/etc/passwd` — `ls` needs it to turn UIDs into names. The password *hashes* are therefore kept in `/etc/shadow`, whose mode `-rw-r-----` lets only `root` and the group `shadow` read it. That is why `cat /etc/shadow` fails for us. Administrators never edit these files by hand; they use `useradd`, `usermod`, `groupadd` and `passwd`.

4 Reading Permissions

4.1 The columns of `ls -l`

```
student@okan-lab:~/lab$ ls -l docs
total 24
```

```
-rw-rw-r-- 1 student student  53 Oct 12 19:38 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 19:38  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38  syllabus.txt
student@okan-lab:~/lab$ ls -ld docs
drwxrwxr-x 2 student student 4096 Oct 12 19:38 docs
```

The columns are: (1) file type and permissions, (2) link count (Week 7, Week 10), (3) owner, (4) group, (5) size in bytes, (6) modification time and (7) the name. Everything except the name comes from the inode; the name comes from the directory entry. The inode stores *numbers* for owner and group; `ls` shows names by looking them up in `/etc/passwd` and `/etc/group`. Note the difference between `ls -l docs` (the files *inside* `docs`) and `ls -ld docs` (the directory itself).

4.2 Decoding the mode string

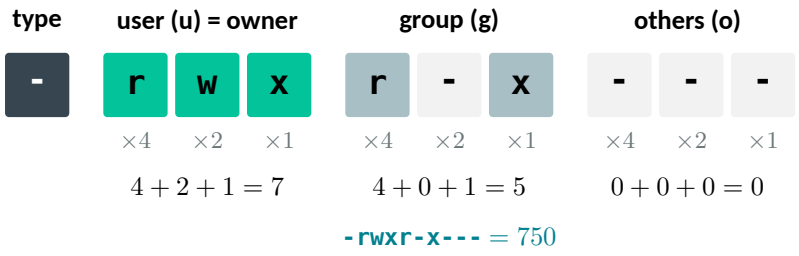


Figure 3: The first character is the type; then three triplets for user, group and others; each bit has weight 4, 2 or 1.

- **Character 1, the type:** `-` regular file, `d` directory, `l` symbolic link, `c`/`b` character/block device.
- **Characters 2–10:** three *triplets*, always in the order user (the owner), group, others; inside each triplet always r, w, x. A letter means the permission is granted, a dash that it is not.

Each triplet has 3 bits, so there are $2^3 = 8$ possible triplets — exactly the eight octal digits 0–7. With weights $r = 4$, $w = 2$, $x = 1$, every triplet is a single digit, and the nine bits are written as three octal digits: `-rwxr-x---` is 750. The command `stat` shows both forms; `stat -c` lets you choose the fields:

```
student@okan-lab:~/lab$ stat -c '%A  %a  %U  %G  %n' docs/syllabus.txt docs src/hello.c
-rw-rw-r--  664  student  student  docs/syllabus.txt
drwxrwxr-x  775  student  student  docs
-rw-rw-r--  664  student  student  src/hello.c
```

4.3 What r, w and x mean

Bit	On a regular file	On a directory
r (4)	read the contents (`cat`, `less`, `cp`)	list the names inside (`ls`)
w (2)	change the contents (editor, >>)	create, delete and rename entries (together with x)
x (1)	run it as a program or script	enter it and reach what is inside (`cd`, paths through it)

Remember from Week 7 that a directory is a table of (name, inode number) pairs. Reading the table means

listing names; writing it means adding or removing entries; x ("search") means following a path through it. Two consequences surprise most beginners: deleting a file is a change to the *directory*, so it needs w on the directory and does not depend on the file's own w bit; and without x on a directory nothing below it can be reached, however generous the files' own permissions are. Section 7 demonstrates both.

4.4 Which class applies? Only one

When a process accesses a file, the kernel does *not* combine the three triplets. It picks exactly one:

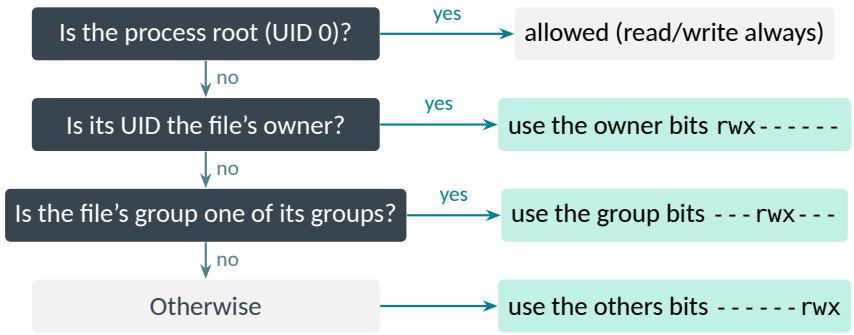


Figure 4: The kernel's access check stops at the first class that matches.

So the owner is judged *only* by the owner triplet, even if the others triplet is more generous. Our demonstration (`code/demo_order.sh`):

```
student@okan-lab:~/lab$ echo "hello" > me.txt
student@okan-lab:~/lab$ chmod 074 me.txt
student@okan-lab:~/lab$ ls -l me.txt
----rwxr-- 1 student student 6 Oct 12 19:38 me.txt
student@okan-lab:~/lab$ cat me.txt
cat: me.txt: Permission denied
student@okan-lab:~/lab$ chmod 644 me.txt
student@okan-lab:~/lab$ cat me.txt
hello
```

With mode 074 = `----rwxr--` everybody else may read, but the owner may not. The owner can still repair the situation, because changing the mode requires *ownership*, not r or w. (For `root`, read and write are always allowed; execute is allowed if at least one x bit is set.)

5 Changing Permissions with `chmod`

Only the owner of a file (or `root`) may change its mode. `chmod` (*change mode*) accepts two notations.

5.1 Symbolic mode

The form is `chmod` *who operator permissions file…*:

- **who:** `u` (user = owner), `g` (group), `o` (others), `a` (all three);
- **operator:** `+` add, `-` remove, `=` set exactly (the unnamed bits of that class are cleared);
- **permissions:** any of `r`, `w`, `x`.

Several clauses are separated by commas *without spaces*: `u=rw,g=r,o=` (the empty `o=` means "no rights for others"). The session below (`code/demo_chmod_sym.sh`) changes `docs/syllabus.txt` step by step:

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+x syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod go-w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod a=r syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-r--r--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u=rw,g=r,o= syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-r----- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+rwx,g+w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw---- 1 student student 578 Oct 12 19:38 syllabus.txt
```

`chmod` prints nothing when it succeeds ("no news is good news"); always check with `ls -l` or `stat`. `+` and `-` leave the other bits untouched, which makes symbolic mode ideal for small changes such as "also let the group write" (`chmod g+w`). Writing `chmod +x file` without a class adds x for every class *not blocked by the umask* (Section 6); `u+x` is clearer.

5.2 Octal mode

In octal mode you give all nine bits at once, one digit per class:

Octal	Binary $r\,w\,x$	Symbolic	Meaning
0	000	`---`	no access
1	001	`--x`	execute only
2	010	`-w-`	write only (rare)
3	011	`-wx`	write and execute (rare)
4	100	`r--`	read only
5	101	`r-x`	read and execute
6	110	`rw-`	read and write
7	111	`rwx`	everything

Because 4, 2 and 1 are powers of two, every digit from 0 to 7 is a *unique* sum of them: this is simply binary, three bits at a time. Two quick checks: an odd digit always contains x; a digit ≥ 4 always contains r.

Converting in both directions

Symbolic → octal. `rwxr-xr--`: `rwx` $= 4+2+1 = 7$, `r-x` $= 4+0+1 = 5$, `r--` $= 4+0+0 = 4$, so the mode is 754. Likewise `rw-rw-r--` $= 664$ and `rw-r-----` $= 640$.
Octal → symbolic. 711: $7 = 4+2+1 \to$ `rwx`, $1 \to$ `--x`, $1 \to$ `--x`, so `rwx--x--x`. 640: $6 = 4+2 \to$ `rw-`, $4 \to$ `r--`, $0 \to$ `---`: `rw-r-----`.
(All conversions in these notes were checked with `code/perm_check.py`.)

Octal	Symbolic	Typical use
644	`rw-r--r--`	ordinary files: everyone reads, only the owner writes
755	`rwxr-xr-x`	programs, scripts and directories that others may use
600	`rw-------`	private files (passwords, SSH keys — `ssh` refuses a key that others can read)
700	`rwx------`	private directories (`~/.ssh`)
640 / 750	`rw-r-----` / `rwxr-x---`	shared with the group, hidden from others
777	`rwxrwxrwx`	everyone may change or delete: almost never correct

Safety: never "fix" an error with `chmod 777`

`chmod 777` (or worse, `chmod -R 777`) makes a Permission-denied message disappear by letting *every* user and every compromised program on the machine change or replace the file. Find the one missing bit and add only that, e.g. `chmod u+x script.sh`. This is the *principle of least privilege*.

The octal session (`code/demo_chmod_oct.sh`) also shows two typical errors:

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ chmod 754 syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr-xr-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 640 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 600 notes_01.txt
student@okan-lab:~/lab/docs$ chmod 644 notes_02.txt
student@okan-lab:~/lab/docs$ chmod 700 ../backup
student@okan-lab:~/lab/docs$ stat -c '%a  %A  %n' syllabus.txt notes_01.txt notes_02.txt ../
    backup
640  -rw-r-----  syllabus.txt
600  -rw-------  notes_01.txt
644  -rw-r--r--  notes_02.txt
700  drwx------  ../backup
student@okan-lab:~/lab/docs$ chmod 854 notes_03.txt
chmod: invalid mode: ‘’854
Try 'chmod --help' for more information.
student@okan-lab:~/lab/docs$ chmod rw notes_03.txt
chmod: invalid mode: ‘’rw
Try 'chmod --help' for more information.
```

The digits 8 and 9 do not exist in octal, and `rw` is neither a number nor a valid symbolic clause (it lacks *who* and an operator: `u+rw`). A recursive change `chmod -R` applies the same mode to files *and* directories; `chmod -R 644 dir` would remove x from every subdirectory and lock you out. With `-R` prefer symbolic mode that only removes rights (`chmod -R go-rwx dir`), or the capital X, which adds x only to directories (and files that already have an x).

5.3 Making a script executable

A newly created file is never executable (Section 6 explains why). The session `code/demo_script.sh`:

```
student@okan-lab:~/lab$ cd scripts
student@okan-lab:~/lab/scripts$ echo '#!/bin/bash' > hello.sh
student@okan-lab:~/lab/scripts$ echo 'echo "Hello, $USER! You are in $PWD"' >> hello.sh
student@okan-lab:~/lab/scripts$ cat hello.sh
#!/bin/bash
```

```
echo "Hello, $USER! You are in $PWD"
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ chmod u+x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
student@okan-lab:~/lab/scripts$ cd ../src
student@okan-lab:~/lab/src$ gcc -Wall -O2 hello.c -o hello
student@okan-lab:~/lab/src$ ls -l hello.c hello
-rwxrwxr-x 1 student student 15960 Oct 12 19:38 hello
-rw-rw-r-- 1 student student    90 Oct 12 19:38 hello.c
student@okan-lab:~/lab/src$ ./hello
Hello from the lab!
```

- Typing `./hello.sh` asks the kernel to *execute* the file (system call `execve`); this requires x. `bash hello.sh` executes `bash`, which only *reads* the file — r is enough.
- `hello.sh` without `./` is "command not found": the shell looks for commands only in the directories listed in PATH, and the current directory is not among them (for security). This is not a permission problem.
- The first line `#!/bin/bash` (the *shebang*) tells the kernel which interpreter runs the script. Week 14 builds on this.
- `gcc` sets the x bits itself: it creates the program with 777, reduced by the umask 002 to 775.

6 Default Permissions: the `umask`

Why are new files 664 and new directories 775? A program that creates a file asks for a mode in its `open()` or `mkdir()` call: editors, `touch` and the shell ask for 666 (`rw-rw-rw-`) for files and 777 for directories. The kernel then *removes* every bit that is set in the process's **umask** (user file-creation mask). The umask is a per-process setting inherited by child processes; it is set at login (on Ubuntu to 002 for normal users, 022 for root) and can be changed with the shell built-in `umask`.

6.1 It clears bits; it does not subtract

Formally,

$$\text{new mode} = \text{requested mode} \ \text{AND} \ (\text{NOT umask}),$$

applied bit by bit. For the common umask 022 this looks like subtraction, $666 - 022 = 644$:

requested	$666 =$	$110\ 110\ 110$
umask	$022 =$	$000\ 010\ 010$
NOT umask		$111\ 101\ 101$
AND		$110\ 100\ 100 = 644 =$ `rw-r--r--`

Where subtraction goes wrong: umask 033

The umask 033 means: take w and x away from group and others. For a *file*:

requested	$666 =$	$110\ 110\ 110$
NOT 033		$111\ 100\ 100$
AND		$110\ 100\ 100 = 644 =$ `rw-r--r--`

Plain subtraction would give $666 - 033 = 633 =$ `rw--wx-wx`: write and execute without read, and x bits on a file that never asked for them. Nonsense! In each digit, $6 - 3 = 3$ "borrows" an x bit that was never set, while the bitwise operation simply leaves an absent bit absent: 6 AND NOT $3 = 110$ AND $100 = 100 = 4$.

For a *directory*, 777 has every bit set, so subtraction happens to agree: $777 - 033 = 744$. In general, subtraction is correct exactly when every bit of the umask is also set in the requested mode. For files (666, no x bits) it therefore fails for every umask containing an x bit, e.g. 027: $666 - 027 = 637$ (wrong) versus the correct 640; and 077: $666 - 077 = 567$ (wrong) versus 600.

umask	**new file** (666)	**new directory** (777)	**typical use**
002	664 `rw-rw-r--`	775 `rwxrwxr-x`	Ubuntu users (private groups)
022	644 `rw-r--r--`	755 `rwxr-xr-x`	root, many servers
027	640 `rw-r-----`	750 `rwxr-x---`	group reads, others nothing
077	600 `rw-------`	700 `rwx------`	fully private
033	644 `rw-r--r--`	744 `rwxr--r--`	the subtraction trap

The real session (`code/demo_umask.sh`) confirms every row:

```
student@okan-lab:~/lab$ mkdir umask_test
student@okan-lab:~/lab$ cd umask_test
student@okan-lab:~/lab/umask_test$ umask
0002
student@okan-lab:~/lab/umask_test$ umask -S
u=rwx,g=rwx,o=rx
student@okan-lab:~/lab/umask_test$ touch f_002
student@okan-lab:~/lab/umask_test$ mkdir d_002
student@okan-lab:~/lab/umask_test$ umask 022
student@okan-lab:~/lab/umask_test$ touch f_022
student@okan-lab:~/lab/umask_test$ mkdir d_022
student@okan-lab:~/lab/umask_test$ umask 077
student@okan-lab:~/lab/umask_test$ touch f_077
student@okan-lab:~/lab/umask_test$ mkdir d_077
student@okan-lab:~/lab/umask_test$ umask 033
student@okan-lab:~/lab/umask_test$ touch f_033
student@okan-lab:~/lab/umask_test$ mkdir d_033
student@okan-lab:~/lab/umask_test$ umask 002
student@okan-lab:~/lab/umask_test$ stat -c '%a  %A  %n' f_* d_*
664  -rw-rw-r--  f_002
644  -rw-r--r--  f_022
644  -rw-r--r--  f_033
600  -rw-------  f_077
775  drwxrwxr-x  d_002
755  drwxr-xr-x  d_022
744  drwxr--r--  d_033
700  drwx------  d_077
```

`umask -S` prints the complement in symbolic form, i.e. what is *allowed*. Because files are requested without x, no umask can ever make a new file executable: that always takes a `chmod`. The umask is a property of the shell process, which is why `umask` (like `cd`) is built into the shell. A change lasts until the shell exits; to make it permanent, add e.g. `umask 027` to `~/.bashrc`. Existing files are never affected.

7 Directory Permissions in Practice

The session `code/demo_dirperm.sh` creates a directory `box` with one file and removes bits from it:

```
student@okan-lab:~/lab$ mkdir box
student@okan-lab:~/lab$ echo "top secret" > box/secret.txt
student@okan-lab:~/lab$ ls -ld box
drwxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ chmod u-x box
student@okan-lab:~/lab$ ls -ld box
drw-rwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
secret.txt
student@okan-lab:~/lab$ ls -l box
ls: cannot access 'box/secret.txt': Permission denied
total 0
-????????? ? ? ? ?            ? secret.txt
student@okan-lab:~/lab$ cd box
bash: cd: box: Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
cat: box/secret.txt: Permission denied
student@okan-lab:~/lab$ chmod u-r,u+x box
student@okan-lab:~/lab$ ls -ld box
d-wxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
ls: cannot open directory 'box': Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
top secret
```

- r **without** x**:** `ls` can read the list of names, but `ls -l` cannot look up the inodes through the directory — hence the question marks — and `cd` and `cat` fail.
- x **without** r**:** the directory cannot be listed, but a file whose name you know can be opened. (A building where you have a key to every room but no list of the rooms.)

In practice, directories have r and x together: `r-x` or `rwx`.

```
student@okan-lab:~/lab$ chmod 755 box
student@okan-lab:~/lab$ chmod 444 box/secret.txt
student@okan-lab:~/lab$ ls -l box
total 4
-r--r--r-- 1 student student 11 Oct 12 19:38 secret.txt
student@okan-lab:~/lab$ echo "more" >> box/secret.txt
bash: box/secret.txt: Permission denied
student@okan-lab:~/lab$ rm box/secret.txt
rm: remove write-protected regular file 'box/secret.txt'? y
student@okan-lab:~/lab$ ls box
```

```
student@okan-lab:~/lab$ chmod 555 box
student@okan-lab:~/lab$ touch box/new.txt
touch: cannot touch 'box/new.txt': Permission denied
student@okan-lab:~/lab$ chmod 755 box
```

The file is read-only (444), so appending fails — but `rm` merely asks a question and then deletes it, because deletion changes the *directory* `box`, which is writable. Only when the directory itself becomes 555 can nothing be added or removed.

> **Safety: `rm` and read-only files**
>
> A read-only file is *not* protected against deletion. `rm -f` even skips the question, and `rm -r` deletes whole trees without a recycle bin (Week 10). To protect files from deletion, remove w from the directory that contains them.

8 Ownership: `chown` and `chgrp`

`chown user file` changes the owner (`chown user:group file` both); `chgrp group file` changes the group; both accept `-R`. The rules on Linux:

- Only `root` may change the owner. Otherwise a user could "give away" files to escape a disk quota or make a file appear to be someone else's.
- The owner may change the group, but only to a group he or she belongs to.

```
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ chown root docs/notes_01.txt
chown: changing ownership of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
chgrp: changing group of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chown $USER docs/notes_01.txt
```

Note the different message: *Operation not permitted* (`EPERM`) means that the operation needs a privilege, whatever the permission bits; *Permission denied* (`EACCES`) means a permission bit is missing.

8.1 Sharing a folder with a group

For the next session the administrator ran `sudo usermod -aG students student` (add, `-a`, to the supplementary group, `-G`, `students`) and the student logged in again — group membership is read at login. (Without `-a`, `usermod -G` *replaces* the user's supplementary groups: a classic administrator mistake.)

```
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student),1010(students)
student@okan-lab:~/lab$ groups
student students
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student students 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ mkdir shared
student@okan-lab:~/lab$ chgrp students shared
```

```
student@okan-lab:~/lab$ chmod 2770 shared
student@okan-lab:~/lab$ ls -ld shared
drwxrws--- 2 student students 4096 Oct 12 19:38 shared
student@okan-lab:~/lab$ touch shared/plan.txt
student@okan-lab:~/lab$ ls -l shared
total 0
-rw-rw-r-- 1 student students 0 Oct 12 19:38 plan.txt
```

Now `chgrp` works. Mode 2770 has a fourth, leading digit: 2 is the **setgid** bit, shown as `s` in the group's x position. On a directory it makes new files inherit the directory's group: `plan.txt` belongs to `students`, not to the creator's primary group. A team folder therefore needs three steps: a common group, `chgrp` to that group, and `chmod 2770`.

9 root and sudo

`root` (UID 0) is not subject to the read and write checks and may change owners, install software and reconfigure the system. Working as root all the time is dangerous: a single typo can destroy the system. Ubuntu therefore has no root password by default; instead, administrators run single commands with `sudo` (*substitute user do*):

- `sudo command` runs *one* command as root;
- it asks for *your own* password (not root's) and remembers it for about 15 minutes per terminal (`sudo -k` forgets it);
- only users permitted in `/etc/sudoers` may use it — on Ubuntu, members of the group `sudo`;
- every use is logged.

Our student before being made an administrator (the password is typed but not shown):

```
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
student is not in the sudoers file.
```

After the administrator added the student to the group `sudo` (a root set-up step):

```
student@okan-lab:~/lab$ groups
student sudo students
student@okan-lab:~/lab$ whoami
student
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
root
student@okan-lab:~/lab$ ls -l docs/notes_02.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38 docs/notes_02.txt
student@okan-lab:~/lab$ chown root docs/notes_02.txt
chown: changing ownership of 'docs/notes_02.txt': Operation not permitted
student@okan-lab:~/lab$ sudo chown root:root docs/notes_02.txt
student@okan-lab:~/lab$ ls -l docs/notes_02.txt
-rw-rw-r-- 1 root root 16 Oct 12 19:38 docs/notes_02.txt
student@okan-lab:~/lab$ echo "more" >> docs/notes_02.txt
bash: docs/notes_02.txt: Permission denied
student@okan-lab:~/lab$ sudo -k
```

This is the only output in these notes where a command runs with root privileges, and it is labelled as such. After `sudo chown root:root`, the mode is still 664, but `student` now falls into the *others* class, which has only r: appending fails.

> **Safety: sudo**
>
> Read every `sudo` command twice before pressing Enter. As root, `sudo rm -r / home/x` (note the space after `/`) tries to delete the whole system. Never paste `sudo` commands from the internet that you do not understand, and never use `sudo` just to get rid of a Permission-denied message in your own files — fix the permissions instead.

10 Special Bits: setuid and the Sticky Bit

Besides the nine permission bits, the mode has three special bits, written as a leading fourth octal digit: setuid $= 4000$, setgid $= 2000$ (seen above), sticky $= 1000$. In total that gives $2^{12} = 4096$ possible modes.

10.1 setuid: how `passwd` can write `/etc/shadow`

```
student@okan-lab:~/lab$ ls -l /usr/bin/passwd /etc/shadow
-rw-r----- 1 root shadow   884 Oct 12 11:50 /etc/shadow
-rwsr-xr-x 1 root root   64152 May 30  2024 /usr/bin/passwd
student@okan-lab:~/lab$ ls -l /usr/bin/sudo /usr/bin/su
-rwsr-xr-x 1 root root  55680 Mar  6  2026 /usr/bin/su
-rwsr-xr-x 1 root root 277936 Mar  2  2026 /usr/bin/sudo
```

Users must be able to change their own password, but `/etc/shadow` is writable only by root. The program `/usr/bin/passwd` is owned by root and has the **setuid** bit (the `s` in the owner's x position, 4755). A process started from a setuid file runs with the *effective* UID of the file's owner — here root — while its *real* UID remains `student`, so `passwd` knows whose password to change. This is privilege granted to a small piece of trusted code, the same idea as the system call in Week 2. A bug in a setuid-root program gives an attacker root, so there are very few of them (`passwd`, `su`, `sudo`, …), and you should never make your own scripts setuid (Linux ignores setuid on scripts anyway).

10.2 The sticky bit on `/tmp`

```
student@okan-lab:~/lab$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 19:38 /tmp
student@okan-lab:~/lab$ stat -c '%a  %A  %n' /usr/bin/passwd /tmp
4755  -rwsr-xr-x  /usr/bin/passwd
1777  drwxrwxrwt  /tmp
student@okan-lab:~/lab$ ls -l /tmp/w11_admin.txt
-rw-rw-rw- 1 root root 12 Oct 12 19:38 /tmp/w11_admin.txt
student@okan-lab:~/lab$ rm /tmp/w11_admin.txt
rm: cannot remove '/tmp/w11_admin.txt': Operation not permitted
student@okan-lab:~/lab$ echo "my notes" > /tmp/w11_mine.txt
student@okan-lab:~/lab$ ls -l /tmp/w11_mine.txt
-rw-rw-r-- 1 student student 9 Oct 12 19:38 /tmp/w11_mine.txt
student@okan-lab:~/lab$ rm /tmp/w11_mine.txt
```

`/tmp` has mode 1777 = `rwxrwxrwt`: everybody may create files there. Without further rules, w on the directory would let everyone delete everyone else's files (Section 7). The **sticky bit** (`t` in the others' x position) adds the rule: in such a directory, only the owner of a file (or of the directory, or root) may delete or rename it. For this demonstration root had created a world-writable file `/tmp/w11_admin.txt`; our student still cannot delete it, but can delete his own file.

11 Command Reference

Command	Purpose	Example
`whoami`, `id`, `groups`	user name; UID, GID, groups	`id`
`getent group G`	one line of the group database	`getent group students`
`ls -l`, `ls -ld`	mode, owner, group (of the directory itself with `-d`)	`ls -ld docs`
`stat -c FORMAT`	`%a` octal, `%A` symbolic, `%U`/`%G` owner/group, `%n` name	`stat -c '%a %A %n' *`
`chmod who±=perm`	change bits symbolically	`chmod u+x,go-w f`
`chmod NNN`	set all nine bits in octal	`chmod 640 f`
`chmod -R`	recursive (prefer symbolic)	`chmod -R go-rwx dir`
`umask`, `umask -S`	show or set the creation mask	`umask 027`
`chown U[:G]`	change owner (root only)	`sudo chown root:root f`
`chgrp G`	change group (to one of your groups)	`chgrp students dir`
`sudo CMD`	run one command as root	`sudo whoami`
`sudo -k`	forget the cached sudo password	`sudo -k`
`usermod -aG G U`	add a user to a group (administrator)	`sudo usermod -aG students ali`

`man` pages give the details (`man chmod`; sections NAME, SYNOPSIS, DESCRIPTION; `/word` searches, `q` quits); `chmod --help` prints a short summary.

12 Common Errors and What They Mean

Message	Cause and fix
`bash: ./hello.sh: Permission denied`	The file has no x bit. `chmod u+x hello.sh` (or run `bash hello.sh`).
`bash: hello.sh: command not found`	Not a permission problem: `.` is not in PATH. Type `./hello.sh`.
`cat: /etc/shadow: Permission denied`	You are in the others class and it has no r. Correct behaviour for system files.
`bash: cd: box: Permission denied`	The directory has no x for you. `chmod u+x box`.
`ls: cannot open directory 'box': Permission denied`	The directory has no r. `chmod u+r box`.
`-????????? ? ? ? ? ? secret.txt`	`ls -l` on a directory with r but no x: names visible, inodes not.
`chown: …: Operation not permitted`	Only root may change the owner; ask the administrator (`sudo chown`).
`chgrp: …: Operation not permitted`	You are not a member of that group (or not the owner).
`chmod: invalid mode: '854'`	8 and 9 are not octal digits; or a symbolic mode without *who*/operator (`rw`).
`rm: remove write-protected regular file?`	The file is read-only, but the directory is writable: answering y deletes it.
`rm: cannot remove '/tmp/…': Operation not permitted`	Sticky bit: not your file.
`student is not in the sudoers file.`	You are not an administrator; `sudo` is refused (and logged).

13 Summary

Week 11 in eight sentences

1. Every file access is a system call, so the kernel (Week 2) can compare the process's UID and GIDs with the owner, group and mode stored in the inode (Week 7).
2. Users and groups have names and numbers; `id` shows them, `/etc/passwd` and `/etc/group` store them, `/etc/shadow` holds the password hashes.
3. The mode string is a type character plus three triplets (user, group, others) of r, w, x; with weights $4, 2, 1$ each triplet is one octal digit, e.g. `-rwxr-x---` $= 750$.
4. Exactly one class applies: owner, else group, else others.
5. On directories r lists, w creates and deletes entries, x enters; deleting a file needs w on the directory, not on the file.
6. `chmod` works symbolically (`u+x`, `go-w`, `a=r`) or in octal (`644`, `755`, `600`); never `777`.
7. The umask clears bits from $666/777$: $002 \to 664/775$, $022 \to 644/755$, and $033 \to 644$ for files (bitwise, not 633).

8. Only root changes owners; `sudo` runs one command as root; setuid (`passwd`) and the sticky bit (`/tmp`) are special bits.

14 Glossary

EACCES / EPERM Error codes behind "Permission denied" (a bit is missing) and "Operation not permitted" (a privilege is missing).

GID Group ID, the number of a group; each user has a primary and possibly supplementary groups.

Mode The type and the 12 permission bits of a file (nine rwx bits plus setuid, setgid, sticky), stored in the inode.

Octal notation Writing each triplet as one digit 0–7 with $r = 4$, $w = 2$, $x = 1$.

root The superuser, UID 0.

setgid On a directory: new files inherit the directory's group (2000).

setuid A program runs with the effective UID of its owner (4000), e.g. `passwd`.

Sticky bit In a shared directory, only a file's owner may delete or rename it (1000), e.g. `/tmp`.

sudo Runs one command as root for users allowed in `/etc/sudoers`.

UID User ID, the number by which the kernel identifies a user.

umask Per-process mask of bits that are cleared from the mode of every newly created file or directory.

15 Lab Sheet: Secure a Project Folder

Work in `~/lab` after running `bash mcmp117_lab_setup.sh`. For each exercise hand in the command(s) you used *and* the `ls -l` or `stat` line that proves the result. Total: 100 points (Part A 20, Part B 48, Part C 32). Model solutions are in Section 18. Extra practice: `ls -l /usr/bin | grep rws` lists the setuid programs on your machine, and `python3 code/perm_check.py` prints a conversion table to check your answers.

Part A --- Who am I? (basic)

1. **(4 pts)** Run `whoami`, `id` and `groups`. Write down your UID, your primary group and the number of groups you belong to.
2. **(5 pts)** Show your own line of `/etc/passwd` (`grep "^$USER:" /etc/passwd`). Name all seven fields. What are your home directory and your login shell?
3. **(5 pts)** Run `ls -l docs/syllabus.txt`. Who may read and who may write the file? Give its mode in octal and check with `stat -c '%a %A'`.
4. **(6 pts)** On paper: convert `rwxr-x---`, `rw-r--r--` and `rwx------` to octal, and 640, 711, 775 to symbolic notation.

Part B --- chmod and umask (intermediate)

5. **(6 pts)** Create the script `scripts/hello.sh` with the two `echo` commands of Section 5.3 (writing into `scripts/hello.sh`). Make it executable for the *owner only* and run it as `./scripts/hello.sh`.
6. **(6 pts)** Using *symbolic* mode only, change `docs/notes_01.txt` from `rw-rw-r--` to `rw-r-----`.
7. **(6 pts)** Using *octal* mode only, make `docs/notes_02.txt` readable and writable by its owner and by nobody else. Verify with `stat`.
8. **(6 pts)** Make `projects/alpha` private: only you may list, enter or change it. Must you also change the files inside? Why (not)?
9. **(6 pts)** `projects/beta`: the group may list and enter it but not change it; others get nothing. Give `projects/beta/README.md` the mode `rw-r-----`.
10. **(6 pts)** Predict the modes of a new file and a new directory with `umask 027`. Verify, then restore `umask 002`.
11. **(6 pts)** Do the same with `umask 033`. Explain, with the binary of 666 and 033, why the file is not 633.
12. **(6 pts)** Create a directory `t12` containing a file `a.txt`. Remove the owner's x from `t12`. Which of `ls t12`, `cd t12`, `cat t12/a.txt` still work? Explain, then restore the bit.

Part C --- Challenge

13. **(7 pts)** Make `docs/notes_03.txt` read-only (444) and delete it. Why does this work? Then make `docs/linux_quotes.txt` read-only, set `docs` to 555 and try `rm -f` again. Explain the difference and restore `docs` to 775.
14. **(7 pts)** Try to give `src/hello.c` to `root` with `chown`, and to the group `students` with `chgrp`. Explain both error messages. Which command would an administrator use?
15. **(8 pts)** Show the modes of `/usr/bin/passwd`, `/etc/shadow` and `/tmp`. Explain the `s` and the `t`. Create a file in `/tmp`, list it and delete it.
16. **(10 pts) Team folder.** Create `projects/team` containing `deploy.sh` (a two-line script that prints `deploying...`), `secret.txt` and `notes.txt`. Requirements: the folder may be entered and listed by the group but not changed; `deploy.sh` may be run by you and by the group; `notes.txt` may be read by the group; `secret.txt` only by you. Others get nothing anywhere. Give the octal modes, set them, show all four with one `stat` command and run the script.

16 Review Questions

Conceptual

1. Why can a user program not simply bypass file permissions? Use the words *system call* and *kernel mode*.
2. What is the difference between a UID and a user name? Which one does the inode store?
3. Why are password hashes stored in `/etc/shadow` and not in `/etc/passwd`?
4. Explain the meaning of r, w and x on a directory. Why is deleting a read-only file possible?

5. A file has mode `----rw-rw-` and you are its owner. Can you read it? Can you make it readable for yourself?
6. When would you use symbolic mode and when octal mode? Why is `chmod -R 644` on a directory tree a bad idea?
7. What is the difference between "Permission denied" and "Operation not permitted"?
8. Why is `passwd` setuid root, and why are setuid programs a security risk?

Numerical

9. Convert to octal: `rwxrwx---`, `r-xr-xr-x`, `rw-------`, `rwxr-xr--`.
10. Convert to symbolic: 604, 751, 440, 2770.
11. A file has mode 664. What is its mode after `chmod o-r,g-w`, and after `chmod a+x` from the original?
12. Compute the modes of a new file and a new directory for the umasks 007, 011 and 077. For which of them does plain subtraction give a wrong answer for the file?
13. How many different permission combinations exist for one class? For all nine bits? Including the three special bits?

Answers to the numerical questions

9. `rwxrwx---` $= 770$; `r-xr-xr-x` $= 555$; `rw-------` $= 600$; `rwxr-xr--` $= 754$.
10. 604 = `rw----r--`; 751 = `rwxr-x--x`; 440 = `r--r-----`; 2770 = `rwxrws---` (setgid shown as `s`).
11. `o-r,g-w`: `rw-rw-r--` $\rightarrow$ `rw-r-----` $= 640$. `a+x`: `rwxrwxr-x` $= 775$ (each digit $+1$, since x was absent everywhere).
12. umask 007: file 666 AND $770 = 660$, directory 770; subtraction gives $666 - 007 = 657$ (wrong). umask 011: file 666 (no x to clear), directory 766; subtraction gives 655 (wrong). umask 077: file 600, directory 700; subtraction gives 567 (wrong). Directories are always right by subtraction because 777 contains every bit. (Checked with `code/perm_check.py`.)
13. One class: $2^3 = 8$. Nine bits: $2^9 = 512$. With setuid, setgid and sticky: $2^{12} = 4096$.

17 Looking Ahead: Week 12

Next week, **Text Processing, Redirection & Pipes**, connects commands together. Every process starts with three open file descriptors, 0 (standard input), 1 (standard output) and 2 (standard error) — the same numbers as in the `write(1, …)` calls of Weeks 2 and 3. We redirect them into files (`>`, `>>`, `2>`, `2>&1`), connect processes with pipes (`|`) and analyse the web-server log `~/lab/logs/access.log` with `grep`, `sort`, `uniq`, `cut` and `wc`. Permissions return immediately: redirecting output into a file you may not write gives exactly the "Permission denied" of this week.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Section 13.4 (Protection, in the File-System Interface chapter), Chapter 17 (Protection) and the security part of

Chapter 20 (The Linux System).

- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 39, "Interlude: Files and Directories" (section on permission bits and access control lists): https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf
  - Security chapter "Access Control": https://pages.cs.wisc.edu/~remzi/OSTEP/security-access.pdf
- D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System", *Communications of the ACM* 17(7), 1974, pp. 365–375 (Section 3.5 describes the protection bits and the set-user-ID bit).

18 Lab Sheet Solutions

All outputs are real (`code/lab_solutions.sh`, captured in `code/out_lab_solutions.txt`); dates and numbers will differ on your machine.

1. `whoami; id; groups` — output exactly as in Section 3.1: UID 1011 (yours will differ), primary group `student`, one group.

2. The command `grep "^$USER:" /etc/passwd` prints `student:x:1011:1013::/home/student:/bin/bash`. Fields: user name, password placeholder `x`, UID, GID, comment (empty), home `/home/student`, shell `/bin/bash`.

3. Owner and group may read and write, others may only read: 664.

```
student@okan-lab:~/lab$ ls -l docs/syllabus.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38 docs/syllabus.txt
student@okan-lab:~/lab$ stat -c '%a  %A' docs/syllabus.txt
664  -rw-rw-r--
```

4. `rwxr-x---` $= 750$, `rw-r--r--` $= 644$, `rwx------` $= 700$; $640 =$ `rw-r-----`, $711 =$ `rwx--x--x`, $775 =$ `rwxrwxr-x`.

5. u+x adds execute for the owner only (`chmod 744` would also be accepted if the original was 644; here it was 664).

```
student@okan-lab:~/lab$ echo '#!/bin/bash' > scripts/hello.sh
student@okan-lab:~/lab$ echo 'echo "Hello, $USER! You are in $PWD"' >> scripts/hello.sh
student@okan-lab:~/lab$ chmod u+x scripts/hello.sh
student@okan-lab:~/lab$ ls -l scripts/hello.sh
-rwxrw-r-- 1 student student 49 Oct 12 19:38 scripts/hello.sh
student@okan-lab:~/lab$ ./scripts/hello.sh
Hello, student! You are in /home/student/lab
```

6. Remove w from the group and r from others (`chmod u=rw,g=r,o=` is also correct).

```
student@okan-lab:~/lab$ chmod g-w,o-r docs/notes_01.txt
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
```

```
-rw-r----- 1 student student 16 Oct 12 19:38 docs/notes_01.txt
```

7. $600 =$ `rw-------`.

```
student@okan-lab:~/lab$ chmod 600 docs/notes_02.txt
student@okan-lab:~/lab$ stat -c '%a  %A  %n' docs/notes_02.txt
600  -rw-------  docs/notes_02.txt
```

8. 700. The files inside need not change: without x on `alpha`, nobody else can reach them.

```
student@okan-lab:~/lab$ chmod 700 projects/alpha
student@okan-lab:~/lab$ ls -ld projects/alpha
drwx------ 4 student student 4096 Oct 12 19:38 projects/alpha
```

9. Directory 750 (group `r-x`), file 640. (The subdirectories still have 775, but others cannot get past `beta`.)

```
student@okan-lab:~/lab$ chmod 750 projects/beta
student@okan-lab:~/lab$ chmod 640 projects/beta/README.md
student@okan-lab:~/lab$ ls -ld projects/beta
drwxr-x--- 4 student student 4096 Oct 12 19:38 projects/beta
student@okan-lab:~/lab$ ls -l projects/beta
total 12
-rw-r----- 1 student student   15 Oct 12 19:38 README.md
drwxrwxr-x 2 student student 4096 Oct 12 19:38 docs
drwxrwxr-x 2 student student 4096 Oct 12 19:38 src
```

10. Prediction: 666 AND NOT $027 = 640$, 777 AND NOT $027 = 750$.

```
student@okan-lab:~/lab$ umask 027
student@okan-lab:~/lab$ touch t10.txt
student@okan-lab:~/lab$ mkdir t10
student@okan-lab:~/lab$ stat -c '%a  %A  %n' t10.txt t10
640  -rw-r-----  t10.txt
750  drwxr-x---  t10
student@okan-lab:~/lab$ umask 002
```

11. File 644, directory 744. Bitwise: $666 = 110\,110\,110$, NOT $033 = 111\,100\,100$, AND $= 110\,100\,100 = 644$. The file never had x bits, so only w is removed from group and others; 633 would wrongly "borrow" x bits.

```
student@okan-lab:~/lab$ umask 033
student@okan-lab:~/lab$ touch t11.txt
student@okan-lab:~/lab$ mkdir t11
student@okan-lab:~/lab$ stat -c '%a  %A  %n' t11.txt t11
644  -rw-r--r--  t11.txt
744  drwxr--r--  t11
student@okan-lab:~/lab$ umask 002
```

12. `ls` still lists the names (r is present), but `cd` and `cat` need x to pass through the directory.

```
student@okan-lab:~/lab$ mkdir t12
student@okan-lab:~/lab$ echo "hello" > t12/a.txt
student@okan-lab:~/lab$ chmod u-x t12
```

```
student@okan-lab:~/lab$ ls t12
a.txt
student@okan-lab:~/lab$ cd t12
bash: cd: t12: Permission denied
student@okan-lab:~/lab$ cat t12/a.txt
cat: t12/a.txt: Permission denied
student@okan-lab:~/lab$ chmod u+x t12
student@okan-lab:~/lab$ cat t12/a.txt
hello
```

13. Deleting changes the directory: with `docs` at 775, the read-only file can be deleted (after a question). With `docs` at 555 the directory cannot be changed, so even `rm -f` fails.

```
student@okan-lab:~/lab$ chmod 444 docs/notes_03.txt
student@okan-lab:~/lab$ rm docs/notes_03.txt
rm: remove write-protected regular file 'docs/notes_03.txt'? y
student@okan-lab:~/lab$ ls docs
'final report.txt'   linux_quotes.txt   notes_01.txt   notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ chmod 444 docs/linux_quotes.txt
student@okan-lab:~/lab$ chmod 555 docs
student@okan-lab:~/lab$ rm -f docs/linux_quotes.txt
rm: cannot remove 'docs/linux_quotes.txt': Permission denied
student@okan-lab:~/lab$ chmod 775 docs
```

14. Output as in Section 8 (here for `src/hello.c`). `chown` needs root; `chgrp` to `students` fails because the user is not a member of that group. An administrator would use `sudo chown root src/hello.c` or `sudo chgrp students src/hello.c` (or add the user to the group).

15. Output as in the two listings of Section 10 (the own file here is `/tmp/w11_ex15.txt`). `s`: setuid, `passwd` runs with root's effective UID so it can update `/etc/shadow`. `t`: sticky bit, in `/tmp` only a file's owner may delete it.

16. Folder 750, `deploy.sh` 750, `notes.txt` 640, `secret.txt` 600. Create the folder and the three files with `mkdir` and `echo` as in Exercise 5, then:

```
student@okan-lab:~/lab$ chmod 750 projects/team projects/team/deploy.sh
student@okan-lab:~/lab$ chmod 600 projects/team/secret.txt
student@okan-lab:~/lab$ chmod 640 projects/team/notes.txt
student@okan-lab:~/lab$ stat -c '%a  %A  %n' projects/team projects/team/*
750  drwxr-x---  projects/team
750  -rwxr-x---  projects/team/deploy.sh
640  -rw-r-----  projects/team/notes.txt
600  -rw-------  projects/team/secret.txt
student@okan-lab:~/lab$ ./projects/team/deploy.sh
deploying...
```